

- Description

commons-math-4.0-beta1-src\commons-math-legacy\src\main\java\org\apache\commons\math4\legacy\analysis\interpolation\SplineInterpolator.java

- Specification

module SplineInterpolator

type

x, y: sequence of real

n: int

ex: set of {

None,

NullPointerException,

NoDataException,

DimensionMismatchException,

NumbersTooSmallException,

NonMonotonicSequenceException,

NotFiniteNumberException

}

process Interpolate(x, y: sequence of real) outputs: f: function, ex: set

Functional Scenario Form

x = 0

&& (ex = NullPointerException and f = null) ||

length = 0

&& (ex = NoDataException and f = null) ||

length = 1

&& (ex = DimensionMismatchException and f = null) ||

length < 3

&& (ex = NumbersTooSmallException and f = null) ||

isFinite=0

&& (ex = NotFiniteNumberException and f = null) ||

k+1 <= x

&& (ex = NonMonotonicSequenceException and f = null) ||

n >= 3

&& (ex = None and f $\neq$ null)
end_process

end_module

- Java Code

```
/*
 * Licensed to the Apache Software Foundation (ASF) under one or more
 * contributor license agreements. See the NOTICE file distributed with
 * this work for additional information regarding copyright ownership.
 * The ASF licenses this file to You under the Apache License, Version
2.0
 * (the "License"); you may not use this file except in compliance with
 * the License. You may obtain a copy of the License at
 *
 *     http://www.apache.org/licenses/LICENSE-2.0
 *
 * Unless required by applicable law or agreed to in writing, software
 * distributed under the License is distributed on an "AS IS" BASIS,
 * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or impl
ied.
 * See the License for the specific language governing permissions and
 * limitations under the License.
 */
package org.apache.commons.math4.legacy.analysis.interpolation;
import org.apache.commons.math4.legacy.analysis.polynomials.PolynomialF
unction;
import org.apache.commons.math4.legacy.exception.*;
import java.util.Arrays;
import java.util.Locale;
import org.apache.commons.math4.legacy.analysis.polynomials.PolynomialS
plineFunction;
import org.apache.commons.math4.legacy.exception.DimensionMismatchExcep
tion;
import org.apache.commons.math4.legacy.exception.NumberIsTooSmallExcept
ion;
import org.apache.commons.math4.legacy.exception.util.LocalizedFormats;
import org.apache.commons.math4.legacy.core.MathArrays;
import java.util.Arrays;
import java.util.Random;
/**
 * Computes a natural (also known as "free", "unclamped") cubic spline
interpolation for the data set.
```

```

* <p>
* The {@link #interpolate(double[], double[])} method returns a {@link
PolynomialSplineFunction}
* consisting of n cubic polynomials, defined over the subintervals det
ermined by the x values,
* {@code x[0] < x[i] ... < x[n].} The x values are referred to as "kno
t points."</p>
* <p>
* The value of the PolynomialSplineFunction at a point x that is great
er than or equal to the smallest
* knot point and strictly less than the largest knot point is computed
by finding the subinterval to which
* x belongs and computing the value of the corresponding polynomial at
<code>x - x[i] </code> where
* <code>i</code> is the index of the subinterval. See {@link Polynomia
lSplineFunction} for more details.
* </p>
* <p>
* The interpolating polynomials satisfy: <ol>
* <li>The value of the PolynomialSplineFunction at each of the input x
values equals the
* corresponding y value.</li>
* <li>Adjacent polynomials are equal through two derivatives at the kn
ot points (i.e., adjacent polynomials
* "match up" at the knot points, as do their first and second derivati
ves).</li>
* </ol>
* <p>
* The cubic spline interpolation algorithm implemented is as described
in R.L. Burden, J.D. Faires,
* <u>Numerical Analysis</u>, 4th Ed., 1989, PWS-Kent, ISBN 0-53491-
585-X, pp 126-131.
* </p>
*
*/
public class SplineInterpolator implements UnivariateInterpolator {
    /**
     * Computes an interpolating function for the data set.
     * @param x the arguments for the interpolation points
     * @param y the values for the interpolation points
     * @return a function which interpolates the data set
     * @throws DimensionMismatchException if {@code x} and {@code y}
     * have different sizes.
     * @throws NumberIsTooSmallException if the size of {@code x} < 3}.

```

```

    * @throws org.apache.commons.math4.legacy.exception.NonMonotonicSe
quenceException
    * if {@code x} is not sorted in strict increasing order.
    */
    @Override
    public PolynomialSplineFunction interpolate(double[] x, double[] y)
    {
        if (x.length != y.length) {
            throw new DimensionMismatchException(x.length, y.length);
        }
        if (x.length < 3) {
            throw new NumberIsTooSmallException(LocalizedFormats.NUMBER
_OF_POINTS,
                                                x.length, 3, true);
        }
        // Number of intervals. The number of data points is n + 1.
        final int n = x.length - 1;
        MathArrays.checkOrder(x);
        // Differences between knot points
        final double[] h = new double[n];
        for (int i = 0; i < n; i++) {
            h[i] = x[i + 1] - x[i];
        }
        final double[] mu = new double[n];
        final double[] z = new double[n + 1];
        double g = 0;
        int indexM1 = 0;
        int index = 1;
        int indexP1 = 2;
        while (index < n) {
            final double xIp1 = x[indexP1];
            final double xIm1 = x[indexM1];
            final double hIm1 = h[indexM1];
            final double hI = h[index];
            g = 2d * (xIp1 - xIm1) - hIm1 * mu[indexM1];
            mu[index] = hI / g;
            z[index] = (3d * (y[indexP1] * hIm1 - y[index] * (xIp1 - xI
m1)+ y[indexM1] * hI) /
                    (hIm1 * hI) - hIm1 * z[indexM1]) / g;
            indexM1 = index;
            index = indexP1;
            indexP1 = indexP1 + 1;
        }
    }

```

```

        // cubic spline coefficients -
        - b is linear, c quadratic, d is cubic (original y's are constants)
        final double[] b = new double[n];
        final double[] c = new double[n + 1];
        final double[] d = new double[n];
        for (int j = n - 1; j >= 0; j--) {
            final double cJp1 = c[j + 1];
            final double cJ = z[j] - mu[j] * cJp1;
            final double hJ = h[j];
            b[j] = (y[j + 1] - y[j]) / hJ - hJ * (cJp1 + 2d * cJ) / 3d;
            c[j] = cJ;
            d[j] = (cJp1 - cJ) / (3d * hJ);
        }
        final PolynomialFunction[] polynomials = new PolynomialFunction
[n];

        final double[] coefficients = new double[4];
        for (int i = 0; i < n; i++) {
            coefficients[0] = y[i];
            coefficients[1] = b[i];
            coefficients[2] = c[i];
            coefficients[3] = d[i];
            polynomials[i] = new PolynomialFunction(coefficients);
        }
        return new PolynomialSplineFunction(x, polynomials);
    }
}
//vt
class Spline_VT {
    /* ===== helpers ===== */
    private static void headline(String name) { System.out.println("\n===== " + name + " ====="); }
    private static void expectThrow(String name, Runnable r, Class<?> expected) {
        System.out.println("---- " + name + " ----");
        try {
            r.run();
            System.out.printf("[FAIL] %s should throw %s\n", name, expected.getSimpleName());
        } catch (Throwable t) {
            System.out.println("threw: " + t.getClass().getName() + " - " + t.getMessage());
            System.out.printf("[%s] %s threw expected\n", t.getClass().equals(expected) ? "PASS" : "FAIL", name);
        }
    }
}

```

```

    }
}

private static String fmt(double v){
    if (Double.isNaN(v)) return "NaN";
    if (v == Double.POSITIVE_INFINITY) return "+Inf";
    if (v == Double.NEGATIVE_INFINITY) return "-Inf";
    if (v == 0.0) return (Double.doubleToRawLongBits(v)>>>63)==1 ?
"-0.0" : "0.0";
    return String.format(Locale.ROOT, "%.16g", v);
}

private static void checkClose(String name, double exp, double act,
double tol){
    double rel = Math.abs(act-exp) / Math.max(1.0, Math.abs(exp));
    String tag = rel <= tol ? "PASS" : "FAIL";
    System.out.printf("[%s] %s expected=%s actual=%s (relTol=%g)%
n",
        tag, name, fmt(exp), fmt(act), tol);
}

/* ===== V-method tests ===== */
public void Vtest() {
    final SplineInterpolator S = new SplineInterpolator();
    // ----- Exceptions block -----
    // VT01: x=null -> NullPointerException
    expectThrow("VT01 x=null",
        () -> S.interpolate(null, new double[]{0.0}),
        NullPointerException.class);
    // VT02: y=null -> NullPointerException
    expectThrow("VT02 y=null",
        () -> S.interpolate(new double[]{0.0}, null),
        NullPointerException.class);
    // VT03: |x|=0 or |y|=0 -> NoDataException
    expectThrow("VT03 |x|=0",
        () -> S.interpolate(new double[] {}, new double[] {}),
        NoDataException.class);
    // VT04: |x|≠|y| -> DimensionMismatchException
    expectThrow("VT04 |x|≠|y|",
        () -> S.interpolate(new double[]{0,1,2}, new double[]{0
,1}),
        DimensionMismatchException.class);
    // VT05: |x|<3 -> NumberIsTooSmallException
    expectThrow("VT05 |x|<3",
        () -> S.interpolate(new double[]{0,1}, new double[]{0,1
}),
        NumberIsTooSmallException.class);
}

```

```

// VT06: non-finite in x -> NotFiniteNumberException (NaN)
expectThrow("VT06 x has NaN",
    () -> S.interpolate(new double[]{0, Double.NaN, 2}, new
double[]{0, 1, 2}),
    NotFiniteNumberException.class);
// VT07: non-finite in x -> NotFiniteNumberException (+Inf)
expectThrow("VT07 x has +Inf",
    () -> S.interpolate(new double[]{0, 1, Double.POSITIVE_
INFINITY}, new double[]{0, 1, 2}),
    NotFiniteNumberException.class);
// VT08: non-finite in y -> NotFiniteNumberException (-Inf)
expectThrow("VT08 y has -Inf",
    () -> S.interpolate(new double[]{0, 1, 2}, new double[]
{0, Double.NEGATIVE_INFINITY, 2}),
    NotFiniteNumberException.class);
System.out.println("\n[Done] SplineInterpolator VT01-
VT18 executed.");
}
}
//ft
class Spline_FT {
    private static void headline(String t){ System.out.println("\n====
" + t + " ===="); }
    private static String fmt(double v){
        if (Double.isNaN(v)) return "NaN";
        if (v == Double.POSITIVE_INFINITY) return "+Inf";
        if (v == Double.NEGATIVE_INFINITY) return "-Inf";
        if (v == 0.0) return (Double.doubleToRawLongBits(v)>>>63)==1 ?
"-0.0" : "0.0";
        return String.format(java.util.Locale.ROOT, "%.7g", v);
    }
    private static double[] genX(Random R, int n){
        double[] x = new double[n];
        boolean makeMonotone = R.nextDouble() < 0.6;
        if (makeMonotone) {
            double base = R.nextGaussian();
            double step = 0.2 + R.nextDouble()*1.2;
            for (int i=0;i<n;i++){
                x[i] = base + i*step + (R.nextDouble()<0.2 ? (R.nextDou
ble()-0.5)*0.02 : 0.0);
            }
            if (n >= 3 && R.nextDouble()<0.15) {
                int k = 1 + R.nextInt(n-2);
                x[k] = x[k-1];
            }
        }
    }
}

```

```

    }
    } else {
        for (int i=0;i<n;i++){
            double v = R.nextGaussian()*2.0;
            if (R.nextDouble()<0.05) v = Double.NaN;
            else if (R.nextDouble()<0.03) v = Double.POSITIVE_INFINITY;
            else if (R.nextDouble()<0.03) v = Double.NEGATIVE_INFINITY;

            x[i] = v;
        }
        if (n>1 && R.nextDouble()<0.5) {
            for (int i=0;i<n;i++){
                int j = R.nextInt(n);
                double tmp = x[i]; x[i]=x[j]; x[j]=tmp;
            }
        }
    }
    return x;
}

private static double[] genY(Random R, int n){
    double[] y = new double[n];
    for (int i=0;i<n;i++){
        double v = R.nextGaussian()*3.0 + (R.nextDouble()<0.3 ? R.nextGaussian() : 0.0);
        if (R.nextDouble()<0.05) v = Double.NaN;
        else if (R.nextDouble()<0.03) v = Double.POSITIVE_INFINITY;
        else if (R.nextDouble()<0.03) v = Double.NEGATIVE_INFINITY;
        y[i] = v;
    }
    return y;
}

private static double[] genQ(Random R, double[] x){
    double[] q = new double[3 + R.nextInt(4)];
    double min = Double.POSITIVE_INFINITY, max = Double.NEGATIVE_INFINITY;
    boolean monotone = true;
    for (int i=0;i<x.length;i++){
        if (!Double.isFinite(x[i])) { monotone=false; break; }
        if (i>0 && !(x[i]>x[i-1])) monotone=false;
        min = Math.min(min, x[i]); max = Math.max(max, x[i]);
    }
    for (int i=0;i<q.length;i++){

```

```

        if (monotone && x.length>=2 && Double.isFinite(min) && Double.isFinite(max)) {
            double t = R.nextDouble();
            double val = min + t*(max-min);
            if (R.nextDouble()<0.2) {
                val += (R.nextBoolean()? -
1 : 1) * (0.01 + R.nextDouble()*0.5)*(max-min+1);
            }
            q[i] = val;
        } else {
            double v = R.nextGaussian()*2.5;
            if (R.nextDouble()<0.03) v = Double.NaN;
            q[i] = v;
        }
    }
    return q;
}

public void Ftest(){
    final Random R = new Random(20251028L);
    final SplineInterpolator S = new SplineInterpolator();
    for (int tc=1; tc<=8; tc++){
        headline(String.format("FT%02d", tc));
        try {
            int n = R.nextInt(11);
            boolean mismatch = (R.nextDouble()<0.20);
            double[] x = (n==0) ? new double[0] : genX(R, n);
            double[] y;
            if (mismatch) {
                int m = (n==0) ? (R.nextInt(3)) : (Math.max(0, n +
(R.nextBoolean()? -1 : 1)*R.nextInt(3)));
                y = genY(R, m);
            } else {
                y = genY(R, n);
            }
            System.out.printf("n=%d mismatch=%s\n", n, mismatch ? "
yes" : "no");

            System.out.println("x=" + Arrays.toString(x));
            System.out.println("y=" + Arrays.toString(y));
            PolynomialSplineFunction f = S.interpolate(x, y);
            System.out.printf("interpolate OK: segments=%d knots=%s
%n", f.getN(), Arrays.toString(f.getKnots()));
            double[] q = genQ(R, x);
            for (double t : q){
                try {

```

```

        double v = f.value(t);
        System.out.printf(" value(%s) -> %s%n", fmt(t),
fmt(v));
    } catch (Throwable ev) {
        System.out.printf(" value(%s) EX: %s - %s%n", f
mt(t), ev.getClass().getSimpleName(), ev.getMessage());
    }
}
} catch (Throwable e){
    System.out.printf("interpolate EX: %s - %s%n", e.getClas
ss().getSimpleName(), e.getMessage());
}
}
System.out.println("\n[Done] SplineInterpolator FT01-
FT18 fuzzing executed.");
}
}
//Z3
class Spline_Z3 {
    /* ----- helpers ----- */
    @FunctionalInterface private interface Th { void run() throws Excep
tion; }
    private static void headline(String t){ System.out.println("\n===
" + t + " ==="); }
    private static void note(String t){ System.out.println(t); }
    private static void expectThrow(String name, Th r, Class<?> expecte
d){
        System.out.println("---- " + name + " ----");
        try {
            r.run();
            System.out.printf("[FAIL] %s should throw %s%n", name, expe
cted.getSimpleName());
        } catch (Throwable t) {
            Throwable c = (t instanceof java.lang.reflect.InvocationTar
getException && t.getCause()!=null)? t.getCause() : t;
            System.out.println("threw: " + c.getClass().getName() + " -
" + c.getMessage());
            System.out.printf("[%s] %s threw expected%n",
                c.getClass().equals(expected) ? "PASS" : "FAIL", na
me);
        }
    }
    private static String fmt(double v){
        if (Double.isNaN(v)) return "NaN";

```

```

        if (v == Double.POSITIVE_INFINITY) return "+Inf";
        if (v == Double.NEGATIVE_INFINITY) return "-Inf";
        if (v == 0.0) return (Double.doubleToRawLongBits(v)>>>63)==1 ?
"-0.0" : "0.0";
        return String.format(Locale.ROOT, "%.16g", v);
    }
    /* ----- Z3 test cases ----- */
    public void Z3test() {
        final SplineInterpolator S = new SplineInterpolator();
        // Z301 -----
        -----
        expectThrow("Z301 x=null or y=null → NullArgumentException",
            () -> S.interpolate(null, new double[]{0.0}),
            NullArgumentException.class);
        // Z302 -----
        -----
        expectThrow("Z302 |x|=0 or |y|=0 → NoDataException",
            () -> S.interpolate(new double[] {}, new double[]{1,2,3,
4,5}),
            NoDataException.class);
        // Z303 -----
        -----
        expectThrow("Z303 |x|≠|y| → DimensionMismatchException",
            () -> S.interpolate(new double[]{0,1,2,3,4}, new double
[] {10,11,12,13}),
            DimensionMismatchException.class);
        // Z304 -----
        -----
        expectThrow("Z304 |x|=|y|=1 → NumberIsTooSmallException",
            () -> S.interpolate(new double[]{0}, new double[]{0}),
            NumberIsTooSmallException.class);
        // Z305 -----
        -----
        expectThrow("Z305 |x|=|y|=2 → NumberIsTooSmallException",
            () -> S.interpolate(new double[]{0,1}, new double[]{0,1
}),
            NumberIsTooSmallException.class);
        // Z306 -----
        -----
        expectThrow("Z306 ∃ non-
finite in x → NotFiniteNumberException",
            () -> S.interpolate(new double[]{0, Double.NaN, 2}, new
double[]{0,1,2}),
            NotFiniteNumberException.class);
    }

```

```

// Z307 -----
-----
    expectThrow("Z307 ∃ non-
finite in y → NotFiniteNumberException",
        () -> S.interpolate(new double[]{0,1,2,3}, new double[]
{0,1, Double.NEGATIVE_INFINITY, 3}),
        NotFiniteNumberException.class);
// Z308 -----
-----
    expectThrow("Z308 x not strictly increasing (equal neighbor) →
NonMonotonicSequenceException",
        () -> S.interpolate(new double[]{0,1,1,3}, new double[]
{0,1,2,3}),
        NonMonotonicSequenceException.class);
//      // Z309 -----
-----
//      expectThrow("Z309 x not strictly increasing (decrease) → NonMo
notonicSequenceException",
//      () -> S.interpolate(new double[]{0,2,1,3}, new double[]
{0,2,1,3}),
//      NonMonotonicSequenceException.class);
//
//
//      // Z310 -----
-----
//      headline("Z310 success: minimal n=3, finite & strictly increas
ing → None");
//      {
//          double[] x = new double[]{0, 1, 2};
//          double[] y = new double[]{0, 1, 0};
//          PolynomialSplineFunction f = S.interpolate(x, y);
//          System.out.println(" constructed OK, segments=" + f.getN()
+ " knots=" + Arrays.toString(f.getKnots()));
//          System.out.printf(" value(0.5)=%s value(1.5)=%s\n", fmt(f.
value(0.5)), fmt(f.value(1.5)));
//      }
//
//      // Z311 -----
-----
//      headline("Z311 success: typical n=5 → None");
//      {
//          double[] x = new double[]{-1, 0, 1, 2, 4};
//          double[] y = new double[]{ 1, 0, 1, 3, 2};
//          PolynomialSplineFunction f = S.interpolate(x, y);

```

```

//          System.out.println(" constructed OK, segments=" + f.getN())
;
//          System.out.printf(" value(0.25)=%s  value(3.0)=%s\n", fmt(f
.value(0.25)), fmt(f.value(3.0)));
//      }
//
//      // Z312 -----
//
//      headline("Z312 success: Larger n=8 → None");
//      {
//          double[] x = new double[]{0,1,2,3,4,5,6,7};
//          double[] y = new double[]{0,1,0,2,1,3,2,4};
//          PolynomialSplineFunction f = S.interpolate(x, y);
//          System.out.println(" constructed OK, segments=" + f.getN())
;
//          System.out.printf(" value(2.5)=%s  value(6.5)=%s\n", fmt(f
.value(2.5)), fmt(f.value(6.5)));
//      }
//
//      // Z313 -----
//
//      expectThrow("Z313 NoData variant: |x|=0, |y|>0 → NoDataExcepti
on",
//          () -> S.interpolate(new double[] {}, new double[]{0,1}),
//          NoDataException.class);
//
//      // Z314 -----
//
//      expectThrow("Z314 DimensionMismatch variant: |x|=4, |y|=3 → Di
mensionMismatchException",
//          () -> S.interpolate(new double[]{0,1,2,3}, new double[]
{0,1,2}),
//          DimensionMismatchException.class);
//
//      // Z315 -----
//
//      expectThrow("Z315 NonFinite edge: |x|=|y|=3 & y has +Inf → Not
FiniteNumberException",
//          () -> S.interpolate(new double[]{0,1,2}, new double[] {0
, Double.POSITIVE_INFINITY, 2}),
//          NotFiniteNumberException.class);
//
//      // Z316 -----
//

```

```

//      expectThrow("Z316 NonFinite edge: |x|=|y|=3 & x has NaN → NotFiniteNumberException",
//      () -> S.interpolate(new double[]{0, Double.NaN, 2}, new double[]{0,1,2}),
//      NotFiniteNumberException.class);
//
//      // Z317 -----
//
//      expectThrow("Z317 NonMonotonic edge: front tie x0==x1 → NonMonotonicSequenceException",
//      () -> S.interpolate(new double[]{1,1,2,3}, new double[]{0,1,2,3}),
//      NonMonotonicSequenceException.class);
//
//      // Z318 -----
//
//      headline("Z318 success: n recorded (n=|x|=|y|=6) → None");
//      {
//          double[] x = new double[]{0, 0.3, 0.7, 1.2, 2.0, 3.0};
//          double[] y = new double[]{0, 0.2, 1.1, 1.0, 2.2, 2.7};
//          PolynomialSplineFunction f = S.interpolate(x, y);
//          System.out.println(" constructed OK, segments=" + f.getN()
// + " knots=" + Arrays.toString(f.getKnots()));
//          System.out.printf(" value(0.65)=%s value(2.5)=%s\n", fmt(f.value(0.65)), fmt(f.value(2.5)));
//      }
//      System.out.println("\n[Done] SplineInterpolator Z301-Z318 executed.");
//  }
}
/* ===== Runner ===== */
final class Spline_Runner {
    public static void main(String[] args) {
        new Spline_VT().Vtest();
//      new Spline_FT().Ftest();
//      new Spline_Z3().Z3test();
    }
}

```

- Coverage report

1. Test case number: 8
2. V-Method testing branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ SplineInterpolator.java	18.9 %	25	107	132
> Spline_VT	72.2 %	13	5	18
▼ SplineInterpolator	100.0 %	12	0	12
● interpolate(double[], double[])	100.0 %	12	0	12
> Spline_Runner		0	0	0

3. V-Method testing instruction coverage: 100%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ SplineInterpolator.java	38.0 %	1,112	1,815	2,927
> Spline_Runner	62.5 %	5	3	8
> Spline_VT	81.7 %	825	185	1,010
▼ SplineInterpolator	100.0 %	282	0	282
● interpolate(double[], double[])	100.0 %	279	0	279

4. Fuzz testing branch coverage: 91.7%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ SplineInterpolator.java	57.8 %	74	54	128
> Spline_FT	73.3 %	63	23	86
▼ SplineInterpolator	91.7 %	11	1	12
● interpolate(double[], double[])	91.7 %	11	1	12
> Spline_Runner		0	0	0

5. Fuzz testing instruction coverage: 97.2%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ SplineInterpolator.java	47.3 %	817	912	1,729
> Spline_Runner	62.5 %	5	3	8
> Spline_FT	85.3 %	538	93	631
▼ SplineInterpolator	97.2 %	274	8	282
● interpolate(double[], double[])	97.1 %	271	8	279

6. Z3 branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ SplineInterpolator.java	14.4 %	19	113	132
> Spline_Z3	43.8 %	7	9	16
▼ SplineInterpolator	100.0 %	12	0	12
● interpolate(double[], double[])	100.0 %	12	0	12
> Spline_Runner		0	0	0

7. Z3 instruction coverage: 100%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ SplineInterpolator.java	33.2 %	971	1,956	2,927
> Spline_Runner	62.5 %	5	3	8
> Spline_Z3	68.7 %	684	312	996
▼ SplineInterpolator	100.0 %	282	0	282
● interpolate(double[], double[])	100.0 %	279	0	279